Resumen de las lecciones 2.3 y 2.4

La cuadricula base del visor y la cuadricula de componentes BIM

IDEA CENTRAL	React proporciona el punto de montaje; That Open UI organiza la interfaz BIM mediante grids, templates, estados y layouts.

1. Leccion 2.3 - cuadricula base del visor

La leccion 2.3 crea el primer **bim-grid** en **ProjectDetailsPage**. Ese grid es la estructura general de la pagina BIM; todavia no es el motor 3D ni el modelo.

El visor necesita mas que un canvas: debe reservar zonas para herramientas, paneles, consultas y el viewport. El grid de That Open permite declarar esas zonas y definir como se distribuyen.

```
React monta <bim-grid>
useEffect espera al montaje
setupGrid() configura el grid
grid.elements declara las zonas
grid.layouts declara las distribuciones
grid.layout = "Main" activa una distribucion
```

En el flujo actual, **setupGrid()** llama a **setupComponents()**. Esta funcion devuelve **{ components, viewport }**: el registro comun de servicios BIM y el elemento visual donde se dibuja el visor.

Concepto	Significado
grid.elements	Diccionario de elementos que el grid puede renderizar.
template	Funcion de That Open UI que devuelve una plantilla HTML mediante BUI.html.
initialState	Datos iniciales que recibe el template; conceptualmente parecido a props iniciales.
grid.layouts	Diccionario de distribuciones posibles del mismo conjunto de elementos.
grid.layout	Nombre del layout activo.

El layout se escribe con una sintaxis parecida a CSS Grid:

```
"sidebar" auto
"componentsGrid" 1fr
/1fr
```

• auto: la fila ocupa el tamaño necesario para su contenido.

• 1fr: la fila o columna ocupa el espacio flexible restante.

• /1fr: existe una única columna que ocupa todo el ancho.

La referencia React y la referencia de That Open cumplen una función parecida, pero pertenecen a sistemas diferentes:

```
React: ref={viewerGrid}
That Open: ${BUI.ref(onCreated)}
```

React crea el elemento raiz. That Open configura los grids internos cuando cada Web Component esta disponible.

2. Leccion 2.4 - cuadricula de componentes BIM

La leccion 2.4 introduce un segundo nivel de grid: **componentsGrid**. Su responsabilidad es organizar el area de trabajo BIM, no la pagina completa.

La separacion queda asi:

```
bim-grid raiz
■■■ sidebar
■■■ componentsGrid
■■■ viewport
■■■ models
■■■ queries
■■■ itemsData
```

El template actual es **componentsGridTemplate**. Recibe el estado { components, viewport } y conecta cada elemento con su template correspondiente.

Elemento	Template conectado	Responsabilidad
viewport	viewportContainerTemplate	Aloja el viewport creado por el motor.
models	modelsPanelTemplate	Muestra modelos y permite cargarlos.
queries	queriesPanelTemplate	Muestra y ejecuta consultas BIM.
itemsData	itemsDataPanelTemplate	Muestra propiedades de la seleccion.

El layout Models actual distribuye el espacio asi:

```
"models viewport itemsData" 1fr
"queries viewport itemsData" 1fr
/22rem 1fr 22rem
```

El viewport ocupa las dos filas porque su nombre se repite. Las columnas laterales tienen 22rem y la columna central ocupa el espacio restante.

3. Evolucion del codigo actual respecto al Word

El material del modulo explica una primera version didactica con header, sidebar y componentsGrid. El codigo actual ya ha avanzado sobre esa base:

• El header del grid raiz ya no se configura.

• El sidebar utiliza gridSidebarTemplate en lugar de ser un placeholder vacio.

• componentsGrid dispone de los layouts Models, Queries y Viewer.

• El evento elementcreated entrega al sidebar la referencia del componentsGrid cuando este ya existe.

• ProjectDetailsPage guarda el Components manager y lo destruye con dispose() al desmontarse.

Por tanto, el Word debe leerse como la explicacion de la construccion progresiva; el repositorio contiene una version posterior y mas completa.

4. Cambio de layouts y comunicacion por eventos

gridSidebarTemplate genera un boton por cada clave existente en grid.layouts. El sidebar no conoce de antemano si habra uno, dos o tres layouts.

```
Object.keys(grid.layouts)
↓
Models | Queries | Viewer
↓
grid.layout = layout seleccionado
```

El evento elementcreated resuelve una dependencia temporal: primero se crea el grid raiz, despues componentsGrid, y finalmente se actualiza el sidebar con la referencia al grid interno.

```
componentsGrid creado
↓
"elementcreated"
↓
grid.updateComponent.sidebar({ grid: componentsGrid })
```

5. No confundir grid, viewport y World

Pieza	Que organiza
bim-grid	La interfaz: paneles, sidebar, layouts y toolbar.
viewport	El elemento visual donde se inserta el renderizado.
World	El entorno 3D: escena, camara, renderer, raycaster y grid 3D.

La jerarquia completa que debes visualizar es:

```
React ProjectDetailsPage
■■■ bim-grid raiz
■■■ gridSidebar
■■■ componentsGrid
■■■ panels
■■■ viewportContainer
■■■ bim-viewport
■■■ World + renderer
■■■ viewportGrid flotante
```

6. Ideas clave para fijar

• El grid no renderiza por si mismo la geometria BIM; organiza la interfaz que contiene el viewport.

• Un template es una funcion de That Open UI que devuelve una plantilla BUI.html.

• initialState es la entrada inicial del template, comparable conceptualmente a props.

• Los nombres de grid.elements y los nombres escritos en el layout deben coincidir exactamente.

• Un grid puede reutilizar los mismos elementos con varios layouts.

• BUI.ref sirve para configurar un Web Component despues de su creacion.

• elementcreated permite conectar componentes cuando se crean de forma asincrona o anidada.

• dispose() es necesario para liberar el motor BIM al desmontar la pagina.

Frase resumen

La leccion 2.3 crea la estructura general del visor y la leccion 2.4 introduce el grid interno que organiza el area BIM. Juntos forman una arquitectura de grids anidados donde React monta, That Open UI compone y el World renderiza.